Chaincode Functions | CLI Commands

This section provides example CLI commands to query and invoke the chaincert chaincode on the chaincert-channel channel. Each command includes a short description of its purpose and expected usage.

---

1. Prerequisites

Run these commands once per terminal session before executing any peer chaincode queries or invokes.

Command

```
# 1) Navigate to the Fabric test network directory
cd ~/fabric-samples/fabric-samples/test-network

# 2) Export Fabric binaries path
export PATH=${PWD}/../bin:$PATH

# 3) Export Fabric configuration path
export FABRIC_CFG_PATH=${PWD}/../config/

# 4) Use the DLSU peer context
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="DLSUMSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/dlsu.example.com/users/Admin@dlsu.example.com/msp
```

```
export CORE_PEER_ADDRESS=localhost:12051
```

Optional common variables for invoke commands:

```
ORDERER_CA="${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem"
ORG1_PEER_TLS="${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt"
DLSU_PEER_TLS="${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt"
```

---

2. Quick Reference: Chaincode Functions

2.1 GET (Query) Operations

Function Name	Description	Arguments
Get_CredentialTemplateOrIssuedCredential	Returns a single credential (template or issued) by its exact ID.	CREDENTIAL_ID
Get_IssuerCredentialTemplates	Lists all credential templates created by a specific issuer.	ISSUER_ID

Get_IssuerIssuedCredentials	Lists all issued credentials (instances) under a specific issuer.	ISSUER_ID
Get_HolderCredentials	Lists all credentials issued to a specific holder/user.	HOLDER_ID
Get_CredentialTemplateOrIssuedCredentialHistory	Returns the full history/audit trail of a credential.	CREDENTIAL_ID
Get_IssuerCredentialsHistory	Returns the full credential history (templates + issued) for a specific issuer.	ISSUER_ID
GetAllVerificationLogs	Returns all verification logs stored in the world state.	None
GetVerificationLogById	Returns a single verification log entry by its log ID.	VERIFICATION_LOG_ID
Check_CredentialExists	Checks if a credential exists on the ledger (true/false).	CREDENTIAL_ID

VerifyCredential	Verifies if a credential is valid (existence, revocation, expiry, status).	CREDENTIAL_ID
GetIssuerStatusLogs	Gets all status-change logs for a specific issuer email.	ISSUER_EMAIL
GetUserStatusLogs	Gets all status-change logs for a specific user email.	USER_EMAIL
GetAllStatusLogs	Gets all issuer + user status-change logs.	None
GetAllIssuerStatusLogs	Gets all issuer-only status-change logs.	None
GetAllUserStatusLogs	Gets all user-only status-change logs.	None
GetIssuerDecisionLogs	Gets issuer approval/rejection decision logs for a specific issuer email.	ISSUER_EMAIL

GetAllIssuerDecisionLogs	Gets all issuer decision logs (approvals + rejections).	None
GetAllIssuerApprovalLogs	Gets all issuer approval logs only.	None
GetAllIssuerRejectionLogs	Gets all issuer rejection logs only.	None
GetAllRootSubordinateLogs	Gets all root subordinate decision logs.	None
GetAllRootSubordinateApprovalLogs	Gets all root subordinate approval logs only.	None
GetAllRootSubordinateRejectionLogs	Gets all root subordinate rejection logs only.	None
GetRootSubordinateLogsByRootAuthority	Gets all subordinate logs for a specific root authority.	ROOT_AUTHORITY_ID
GetRootSubordinateApprovalLogsByRootAuthority	Gets approved-only subordinate logs for a specific root authority.	ROOT_AUTHORITY_ID

GetRootSubordinateRejectionLogsByRootAuthority	Gets rejected-only subordinate logs for a specific root authority.	ROOT_AUTHORITY_ID
GetAdminPasswordChangeLogs	Gets password-change logs for a specific admin account.	ADMIN_EMAIL

2.2 CREATE / INVOKE (Write) Operations

Function Name	**Description**	**Arguments**
InitLedger	Initializes the chaincode ledger (no seed data in this version).	None
CreateCredentialTemplate	Creates a new credential template (blueprint) for issuing credentials.	TEMPLATE_ID, ISSUER_ID, CREDENTIAL_TYPE, CREDENTIAL_NAME, DESCRIPTION, METADATA_JSON, SKILLS_JSON
IssueCredential	Issues a credential instance to a holder based on an existing template.	TEMPLATE_ID, HOLDER_ID, EXPIRY_DATE

RevokeCredential	Revokes a credential (sets status to revoked) and stores a rejection reason.	CREDENTIAL_ID, REJECTION_REASON
ExpireCredential	Marks a credential as expired on-chain.	CREDENTIAL_ID
LogVerification	Creates a verification audit log entry for a credential verification event.	CREDENTIAL_ID, VERIFIER_NAME, VERIFIER_EMAIL, VERIFICATION_RESULT_JSON
LogIssuerStatusChange	Logs enable/disable/status changes for an issuer.	ISSUER_EMAIL, PREVIOUS_STATUS, NEW_STATUS, REASON
LogUserStatusChange	Logs enable/disable/status changes for a user.	USER_EMAIL, PREVIOUS_STATUS, NEW_STATUS, REASON
LogIssuerApproval	Logs an approval decision for an issuer.	ISSUER_ID, ISSUER_EMAIL, ORG_NAME, REASON
LogIssuerRejection	Logs a rejection decision for an issuer.	ISSUER_ID, ISSUER_EMAIL, ORG_NAME, REASON

LogRootSubordinate Approval	Logs that a subordinate issuer was approved by a root authority.	ISSUER_ID, ISSUER_EMAIL, ORG_NAME, ROOT_AUTHORITY_ID, ROOT_AUTHORITY_NAME, REASON
LogRootSubordinate Rejection	Logs that a subordinate issuer was rejected by a root authority.	ISSUER_ID, ISSUER_EMAIL, ORG_NAME, ROOT_AUTHORITY_ID, ROOT_AUTHORITY_NAME, REASON
LogAdminChangePassword	Logs an admin password change or similar security-related action.	ADMIN_EMAIL, CHANGE_TYPE, REASON

---

3. Section 1: GET Operations (Read / Query)

General pattern for queries:

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["FUNCTION_NAME","ARG1","ARG2", "..."]}'
```

3.1 Get Single Credential

Get a specific credential (template or issued) by its exact ID.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredential","CREDENTIAL_ID"]}'
```

3.2 Get Credential Templates (by Issuer)

Get all credential templates created by a specific issuer.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_IssuerCredentialTemplates","ISSUER_ID"]}'
```

3.3 Get Issued Credentials (by Issuer)

Get all issued (non-template) credentials under a specific issuer.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_IssuerIssuedCredentials","ISSUER_ID"]}'
```

3.4 Get Holder Credentials

Get all credentials issued to a specific holder.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_HolderCredentials","HOLDER_ID"]}'
```

3.5 Get Credential History

Get full history / audit trail of a specific credential.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredentialHistory","CREDENTIAL_ID"]}'
```

3.6 Check Credential Exists

Check if a credential exists on the ledger (returns true/false).

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Check_CredentialExists","CREDENTIAL_ID"]}'
```

3.7 Verify Credential

Verify if a credential is valid (checks existence, revocation, expiry, status).

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["VerifyCredential","CREDENTIAL_ID"]}'
```

3.8 Get Issuer Credentials History

Get the full blockchain audit trail of all credentials (templates and issued) under a specific issuer.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_IssuerCredentialsHistory","ISSUER_ID"]}'
```

3.9 Get All Verification Logs

Get all verification log entries stored in the world state.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllVerificationLogs"]}'
```

3.10 Get Verification Log by ID

Get a single verification log entry by its ID (for example, vlog_<txId>).

Command

```
peer chaincode query \
```

```
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetVerificationLogById","VERIFICATION_LOG_ID"]}'
```

3.11 Get Issuer Status Logs

Get all status-change logs for a specific issuer email.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetIssuerStatusLogs","ISSUER_EMAIL"]}'
```

3.12 Get User Status Logs

Get all status-change logs for a specific user email.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetUserStatusLogs","USER_EMAIL"]}'
```

3.13 Get All Status Logs

Get all issuer + user status-change logs.

Command

```
peer chaincode query \
  -C chaincert-channel \
```

```
  -n chaincert \
  -c '{"Args":["GetAllStatusLogs"]}'
```

3.14 Get All Issuer Status Logs

Get all issuer-only status-change logs.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllIssuerStatusLogs"]}'
```

3.15 Get All User Status Logs

Get all user-only status-change logs.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllUserStatusLogs"]}'
```

3.16 Get Issuer Decision Logs (by Email)

Get issuer approval/rejection logs for a specific issuer email.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
```

```
  -c '{"Args":["GetIssuerDecisionLogs","ISSUER_EMAIL"]}'
```

3.17 Get All Issuer Decision Logs

Get all issuer decision logs.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllIssuerDecisionLogs"]}'
```

3.18 Get All Issuer Approval Logs

Get only issuer approval logs.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllIssuerApprovalLogs"]}'
```

3.19 Get All Issuer Rejection Logs

Get only issuer rejection logs.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllIssuerRejectionLogs"]}'
```

3.20 Get All Root Subordinate Logs

Get all root subordinate decision logs.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllRootSubordinateLogs"]}'
```

3.21 Get All Root Subordinate Approval Logs

Get all root subordinate approval logs only.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllRootSubordinateApprovalLogs"]}'
```

3.22 Get All Root Subordinate Rejection Logs

Get all root subordinate rejection logs only.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllRootSubordinateRejectionLogs"]}'
```

3.23 Get Root Subordinate Logs by Root Authority

Get all root subordinate logs for a specific root authority.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetRootSubordinateLogsByRootAuthority","ROOT_AUTHORITY_ID"]}'
```

3.24 Get Root Subordinate Approval Logs by Root Authority

Get approved-only root subordinate logs for a specific root authority.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c
'{"Args":["GetRootSubordinateApprovalLogsByRootAuthority","ROOT_AUTHORITY_ID"]}'
```

3.25 Get Root Subordinate Rejection Logs by Root Authority

Get rejected-only root subordinate logs for a specific root authority.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c
'{"Args":["GetRootSubordinateRejectionLogsByRootAuthority","ROOT_AUTHORITY_ID"]}'
```

3.26 Get Admin Password Change Logs

Get admin password change logs for a specific admin email.

Command

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAdminPasswordChangeLogs","ADMIN_EMAIL"]}'
```

---

4. Section 2: CREATE Operations (Write / Invoke)

General pattern for invoke commands (with TLS and multiple peers):

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["FUNCTION_NAME","ARG1","ARG2", "..."]}'
```

4.1 Create Credential Template

Create a new credential template (blueprint for issuing credentials).

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["CreateCredentialTemplate","TEMPLATE_ID","ISSUER_ID","CREDENTIAL_TYPE","CREDENTIAL_NAME","DESCRIPTION","METADATA_JSON","SKILLS_JSON"]}'
```

4.2 Issue Credential

Issue a credential instance to a specific holder based on an existing template.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
```

```
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["IssueCredential","TEMPLATE_ID","HOLDER_ID","EXPIRY_DATE"]}'
```

4.3 Revoke Credential

Revoke a credential (sets status to revoked) and store a rejection reason.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["RevokeCredential","CREDENTIAL_ID","REJECTION_REASON"]}'
```

4.4 Expire Credential

Mark a credential as expired on-chain.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
```

```
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c '{"Args":["ExpireCredential","CREDENTIAL_ID"]}'
```

4.5 Initialize Ledger

Initialize the chaincode (no seed data).

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["InitLedger"]}'
```

4.6 Create Verification Log

Create a verification audit log entry for a credential verification event.

Command

```
peer chaincode invoke \
```

```
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["LogVerification","CREDENTIAL_ID","VERIFIER_NAME","VERIFIER_EMAIL","
{\"valid\":true,\"reason\":\"ok\"}"]}'
```

4.7 Log Issuer Status Change

Log issuer enable/disable/status change.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
```

```
  -c
'{"Args":["LogIssuerStatusChange","ISSUER_EMAIL","PREVIOUS_STATUS","NEW_STAT
US","REASON"]}'
```

4.8 Log User Status Change

Log user enable/disable/status change.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["LogUserStatusChange","USER_EMAIL","PREVIOUS_STATUS","NEW_STATUS",
"REASON"]}'
```

4.9 Log Issuer Approval

Log issuer approval decision.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
```

```
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["LogIssuerApproval","ISSUER_ID","ISSUER_EMAIL","ORG_NAME","REASON"]
}'
```

4.10 Log Issuer Rejection

Log issuer rejection decision.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
```

```
  -c
'{"Args":["LogIssuerRejection","ISSUER_ID","ISSUER_EMAIL","ORG_NAME","REASON"]
}'
```

4.11 Log Root Subordinate Approval

Log subordinate approval by root authority.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["LogRootSubordinateApproval","ISSUER_ID","ISSUER_EMAIL","ORG_NAME","
ROOT_AUTHORITY_ID","ROOT_AUTHORITY_NAME","REASON"]}'
```

4.12 Log Root Subordinate Rejection

Log subordinate rejection by root authority.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
```

```
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["LogRootSubordinateRejection","ISSUER_ID","ISSUER_EMAIL","ORG_NAME","
ROOT_AUTHORITY_ID","ROOT_AUTHORITY_NAME","REASON"]}'
```

4.13 Log Admin Password Change

Log admin password change.

Command

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "${ORDERER_CA}" \
  -C chaincert-channel \
  -n chaincert \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "${ORG1_PEER_TLS}" \
  --peerAddresses localhost:12051 \
  --tlsRootCertFiles "${DLSU_PEER_TLS}" \
  -c
'{"Args":["LogAdminChangePassword","ADMIN_EMAIL","CHANGE_TYPE","REASON"]}'
```